

Строки

Строки в Python используются когда надо работать с текстовыми данными.

Создание строки. Для создания строк, мы используем парные кавычки `' '` или `" "`:

```
s1 = 'Python'
```

```
s2 = "Pascal"
```

Считывание строки. Для считывания текстовых данных в строковую переменную, мы используем функцию `input()`:

```
s = input() # считали текст
```

```
num = int(input()) # считали текст и преобразовали его в целое число
```

Пустая строка. Для создания пустой строки, мы пишем `s = ''` или `s = ""`.
Пустая строка – это аналог числа 0.

Длина строки. Для определения длины строки (количества символов), мы используем встроенную функцию `len()`:

```
s = 'Hello'
```

```
n = len(s) # значение переменной равно 5
```

```
print(n)
```

Конкатенация и умножение на число. Операторы `+` и `*` можно использовать для строк. Оператор `+` сцепляет две и более строк. Это называется конкатенацией строк. Оператор `*` повторяет строку указанное количество раз.

Выражение	Результат
<code>'AB' + 'cd'</code>	<code>'ABcd'</code>
<code>'A' + '7' + 'B'</code>	<code>'A7B'</code>
<code>'Hi' * 4</code>	<code>'HiHiHiHi'</code>

Оператор принадлежности in. С помощью оператора `in`, мы можем проверять, находится ли одна строка в составе другой. То есть, является ли одна строка подстрокой другой:

```
s = 'All you need is love'
```

```
if 'love' in s:
```

```
print('🐍')

else:

    print('🐍')
```

Так как строка `s` содержит подстроку `'love'`, то будет выведен смайлик 🐍.

В Python можно использовать смайлики [emoji](#) 🐍

Индексация строк

Очень часто бывает необходимо обратиться к конкретному символу в строке. Для этого в Python используются квадратные скобки `[]`, в которых указывается индекс (номер) нужного символа в строке.

Пусть `s = 'Python'`. Таблица ниже, показывает как работает индексация:

Выражение	Результат	Пояснение
<code>s[0]</code>	P	первый символ строки
<code>s[1]</code>	y	второй символ строки
<code>s[2]</code>	t	третий символ строки
<code>s[3]</code>	h	четвертый символ строки
<code>s[4]</code>	o	пятый символ строки
<code>s[5]</code>	n	шестой символ строки

Обратите внимание первый символ строки равен `s[0]`, а не `s[1]`. В Python индексация начинается с `0`, по аналогии с функцией `range(n)`, которая генерировала последовательность натуральных чисел от `0` до `n - 1`.

В отличие от многих языков программирования, в Python есть возможность работы с отрицательными индексами. Если первый символ строки имеет индекс `0`, то последнему элементу присваивается индекс `-1`.

Выражение	Результат	Пояснение
<code>s[-6]</code>	P	первый символ строки

Выражение	Результат	Пояснение
<code>s[-5]</code>	y	второй символ строки
<code>s[-4]</code>	t	третий символ строки
<code>s[-3]</code>	h	четвертый символ строки
<code>s[-2]</code>	o	пятый символ строки
<code>s[-1]</code>	n	шестой символ строки

Таким образом, получаем

Положительные индексы	0	1	2	3	4	5
Строка	P	y	t	h	o	n
Отрицательные индексы	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5
P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

Частая ошибка у начинающих программистов — обращение по несуществующему индексу в строке.

Например, если `s = 'Python'`, и мы попытаемся обратиться к `s[17]`, то мы получим ошибку:

```
IndexError: string index out of range
```

Ошибка возникает, поскольку строка содержит всего 6 символов.

Обратите внимание: если длина строки `s` равна `len(s)`, то при положительной нумерации слева направо, последний элемент имеет индекс равный `len(s) - 1`, а при отрицательной индексации справа налево, первый элемент имеет индекс равный `-len(s)`.

Итерирование строк

Очень часто нужно просканировать всю строку целиком, обрабатывая каждый ее символ. Для этого удобно использовать цикл `for`. Напишем программу, которая выводит каждый символ строки на отдельной строке:

```
s = 'abcdef'

for i in range(len(s)):

    print(s[i])
```

Результатом выполнения такой программы будут строки:

```
a
b
c
d
e
f
```

Мы передаем в функцию `range()` длину строки `len(s)`. В нашем случае длина строки `s`, равна 6. Таким образом, вызов функции `range(len(s))` имеет вид `range(6)` и переменная цикла `i` последовательно перебирает все значения от 0 до 5. Это означает, что выражение `s[i]` последовательно вернет все символы строки `s`. Такой способ итерации строки удобен, когда нам нужен не только сам элемент `s[i]`, но и его индекс `i`.

Если нам не нужен индекс самого символа, то мы можем использовать более короткий способ итерации:

```
s = 'abcdef'

for c in s:

    print(c)
```

Этот цикл пройдет по строке `s`, придавая переменной цикла `c` значение каждого символа (!) в отличие от предыдущего цикла, в котором переменная цикла «бегала» по индексам строки.

Обратите внимание на обозначение переменных цикла. В первом цикле мы используем имя `i`, что соответствует стандартной идеологии наименования переменных цикла. Во втором цикле, мы назвали переменную буквой `c` – первая буква слова char (символ).